

Especificação da Linguagem .CORA

Curso: Engenharia da Computação

Disciplina: Compiladores

Equipe: Álex de Souza Rios Lima, Otavio Costa de Oliveira, Pedro Antonio Mota de Araújo e Rafael Figueiredo de Souza

1. Introdução

A CORA é uma linguagem de programação procedural, fortemente tipada, com sintaxe inspirada em C e Java. Ela foi projetada para fins didáticos, abrangendo as fases de análise léxica e sintática de um compilador, permitindo a manipulação de dados numéricos, lógicos e cadeias de caracteres.

2. Tipos de Dados

A linguagem suporta três tipos básicos de dados:

- `int`: Números inteiros de 32 bits.
- `float`: Números de ponto flutuante (decimais).
- `string`: Sequência de caracteres delimitada por aspas duplas (ex: `"texto"`).

3. Palavras-Chave (Keywords)

As seguintes palavras são reservadas e não podem ser usadas como identificadores:

`int`, `float`, `string`, `if`, `else`, `while`, `for`, `switch`, `case`, `default`, `func`, `proc`,
`return`, `break`, `continue`.

4. Operadores e Expressões

4.1 Operadores Aritméticos

Suporta expressões matemáticas com e sem parênteses, seguindo a precedência padrão:

- `+` (Soma)
- (Subtração)
- (Multiplicação)
- `/` (Divisão)
- `%` (Resto da divisão/Módulo)

4.2 Operadores Lógicos

Utilizados para construir condições complexas:

- `&&` (E lógico)
- `||` (OU lógico)
- `!` (Negação)

4.3 Operadores de Comparação

- `>` (Maior que)
- `<` (Menor que)
- `>=` (Maior ou igual a)
- `<=` (Menor ou igual a)
- `==` (Igual a)
- `!=` (Diferente de)

5. Estruturas de Controle de Fluxo

5.1 Condicional (if/else)

Permite a execução de blocos caso a condição seja satisfeita ou o uso do `else` para o caso contrário.

```
if (x > 10 && y == 0) {
    // bloco de código
} else {
    // bloco alternativo
}
```

5.2 Múltipla Escolha (switch/case)

Implementa o controle de fluxo para múltiplas condições baseadas em uma variável.

```
switch (opcao) {  
    case 1:  
        // código  
        break;  
    default:  
        // código padrão  
}
```

6. Estruturas de Repetição (Loops)

6.1 Loop sem Contador (while)

```
while (condicao) {  
    // bloco  
}
```

6.2 Loop com Contador (for)

```
for (int i = 0; i < 10; i = i + 1) {  
    // bloco  
}
```

6.3 Controle de Interrupção

- **break**: Interrompe a execução do loop imediatamente.
- **continue**: Pula para a próxima iteração do loop.

7. Modularização (Funções e Procedimentos)

- **Procedimentos (**proc**)**: Blocos de código executáveis que não retornam valor.
- **Funções (**func**)**: Blocos de código que obrigatoriamente retornam um valor através da palavra-chave **return**.

Exemplo:

```
func somar(int a, int b) {  
    return a + b;  
}  
  
proc aviso() {  
    // apenas executa ação  
}
```

8. Comentários

A linguagem permite documentação no código-fonte através de:

- `//`: Comentário de linha única.
- `/* ... */`: Comentário de múltiplas linhas.

9. Regras de Declaração e Atribuição

- Toda variável deve ser declarada com um tipo seguido de um identificador e finalizada com ponto e vírgula `;`.
- A atribuição utiliza o símbolo `=`.
- Exemplo: `int contador = 0;`

10. Exemplo de Código Completo na Linguagem

```
/*  
    Programa Exemplo em UPL  
    Calcula média e verifica status  
*/  
  
func calcularMedia(float n1, float n2) {  
    return (n1 + n2) / 2;  
}  
  
proc principal() {  
    float nota1 = 7.5;
```

```

float nota2 = 8.0;
float media = calcularMedia(nota1, nota2);

if (media >= 7.0 || media == 10.0) {
    string status = "Aprovado";
} else {
    string status = "Reprovado";
}

for (int i = 0; i < 3; i = i + 1) {
    if (i == 1) {
        continue;
    }
}
}

```